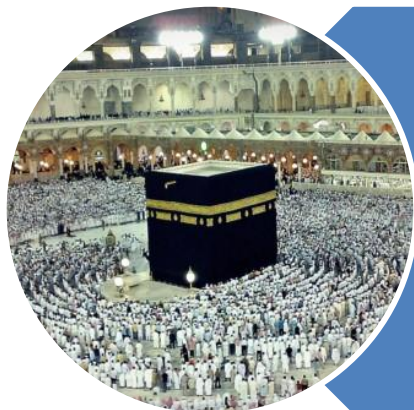


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Recursion

Introduction

C

P

C

S

2

0

4

3

Recursion - Introduction

- Problem Solving Strategy
- In plain English:
 - Recursion: the process a procedure goes through, when one of the steps of the procedure involves rerunning the entire procedure
 - Example: say that some procedure has 4 steps
 - The 3rd step instructs you to run the entire procedure again
 - Each time you get to the third step, you have to start a new
 - This goes on, potentially, infinitely
 - And this is an example of Recursion

C

P

C

S

2

0

4

4

Iterative **vs** Recursion

- Repeat statement(s) using loops e.g “*for*”
- Iterative Algorithm
- Loop may execute infinitely
- Bottom – up approach for problem solving
- Repeat method/function by calling itself.
- Recursive Algorithm
- May execute without termination
- Top- down approach for problem solving

Recursion - Introduction

- From the programming perspective:
 - A recursive method is one that contains a call to its own self
 - Example: we know that we are allowed to call **method B** from **method A** and to call **method A** from **method B**
 - This is not Recursion
 - Also, you are allowed to call method A ; within method A!
 - This is recursion
- Note:
 - This could go on for infinity as method A keeps calling method A
 - So we must have a way to exit the method!

"No Bonus" marks for this course anymore

C

P

C

S

2

0

4

6

Recursion – Example (without Termination)

- Recursion executing infinitely
- Same as executing infinite loop

```
public class printExample {  
    public static void printExample() {  
        System.out.println("Recursion WITHOUT a stopping case.");  
        printExample();  
    }  
  
    public static void main(String[] args) {  
        printExample();  
    }  
}
```

C

P

C

S

2

0

4

Recursion - Introduction

- Programming Perspective
 - Used to solve **large** problems
 - Reduce **larger problem** to **smaller ones of same form**
 - It is a method that **invokes itself**
 - Preferably should have **termination condition** to avoid infinite execution
 - Only solves the **base/termination case**.

Recursion

Example 01

(Printing Count)

Printing Values – Iterative Sol

- Printing from 10 to 01
- Initialized with 10

```
public class countdown {  
    public static void main(String[] args) {  
        int i;  
        for (i = 10; i > 0; --i)  
            System.out.print(i + " ");  
        System.out.println("Blast Off!");  
    }  
}
```

10! 9! 8! 7! 6! 5! 4! 3! 2! 1!

Blast Off

C

P

C

S

2

0

4

10

Printing Values – Recursive Sol

```
public class countdown {
    public static void countdown(int n) {
        if (n==0)
            System.out.println("\nBLAST OFF!");
        else {
            System.out.println(n + "!");
            countdown(n-1);
        }
    }
    static void main(String[] args) {
        countdown(10);
    }
}
```

10! 9! 8! 7! 6! 5! 4! 3! 2! 1!

Blast Off

"No Bonus" marks for this course anymore

C

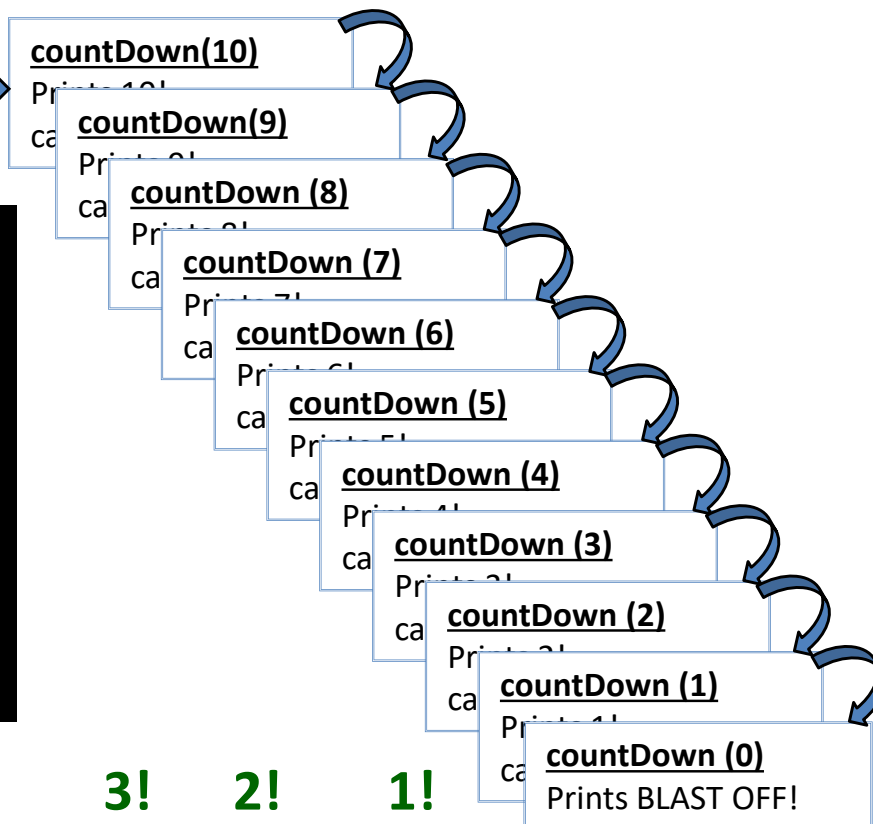
P

C

Printing Values – Recursive Sol

```
static void main(String[] args) {
    countDown(10);
}
```

```
public class countDown {
    public static void countDown(int n) {
        if (n==0)
            System.out.println("\nBLAST OFF!");
        else {
            System.out.println(n + "!");
            countDown(n-1);
        }
    }
    static void main(String[] args) {
        countDown(10);
    }
}
```



10! 9! 8! 7! 6! 5! 4! 3! 2! 1!

Blast Off

“No Bonus” marks for this course anymore

C

P

C

S

2

0

4

Printing Values – Iterative Sol

- Printing from 1 to 10
- Initialize i with 1

```
public class countDown {  
    public static void main(String[] args) {  
        int i;  
        for (i = 1; i <= 10; i++)  
            System.out.print(i + "!" );  
            System.out.println("Blast Off!" );  
        }  
    }  
}
```

Printing values from 10 to 1

1! 2! 3! 4! 5! 6! 7! 8! 9! 10!

Blast Off

"No Bonus" marks for this course anymore

C

P

C

S

2

0

4

Printing Values – Recursive Sol

```
public class countDown {  
    public static void countDown(int n) {  
        if (n==11)  
            System.out.println("\nBLAST OFF!");  
        else {  
            System.out.println(n + "!");  
            countDown(n+1);  
        }  
    }  
    static void main(String[] args) {  
        countDown(1);  
    }  
}
```

1! 2! 3! 4! 5! 6! 7! 8! 9! 10!

Blast Off

C

P

C

Printing Values – Recursive Sol

```
static void main(String[] args) {
    countDown(1);
}
```

```
public class countDown {
    public static void countDown(int n) {
        if (n==11)
            System.out.println("\nBLAST OFF!");
        else {
            System.out.println(n + "!");
            countDown (n+1) ;
        }
    }
    static void main(String[] args) {
        countDown (1) ;
    }
}
```

countDown(1)

countDown(2)

countDown (3)

countDown (4)

countDown (5)

countDown (6)

countDown (7)

countDown (8)

countDown (9)

countDown (10)

countDown (11)

Prints BLAST OFF!

1! 2! 3! 4! 5! 6! 7! 8! 9! 10!

Blast Off

Printing Values – Iterative Sol

- Initialize from 10
- Print from 1 to 10

Change the formula

```
public class countDown {
    public static void main(String[] args) {
        int i;
        for (i = 10; i > 0; --i)
            System.out.print(11-i+ " ");
        System.out.println("Blast Off!");
    }
}
```

Printing values from 10 to 1

1! 2! 3! 4! 5! 6! 7! 8! 9! 10!

Blast Off

C

P

C

S

2

0

4

Printing Values – Recursive Sol

Changing the formula

```
public class countdown {
    public static void countdown(int n) {
        if (n==0)
            System.out.println("\nBLAST OFF!");
        else {
            System.out.println( 11 - n + "!" );
            countdown(n-1);
        }
    }
    static void main(String[] args) {
        countdown(10);
    }
}
```

1! 2! 3! 4! 5! 6! 7! 8! 9! 10!

Blast Off

C

P

C

S

2

0

4

Printing Values – Recursive Sol

Without Changing the formula

```
public class countdown {
    public static void countdown(int n) {
        if (n > 0)

            System.out.println(n + "!");
            countdown(n-1);
        }
    }
    static void main(String[] args) {
        countdown(10);
    }
}
```

10! 9! 8! 7! 6! 5! 4! 3! 2! 1!

“No Bonus” marks for this course anymore

C

P



2

0

4

Printing Values – Recursive Sol

Without Changing the formula

```
public class countdown {
    public static void countdown(int n) {
        if (n > 0)

            countdown(n-1);
            System.out.println(n + "!");
        }
    }

    static void main(String[] args) {
        countdown(10);
    }
}
```

1! 2! 3! 4! 5! 6! 7! 8! 9! 10!

C

P

C

S

2

0

4

Recursion - Summary

- **Invoke** the recursive method by calling it with **initial value**. i.e. `countdown(10)`, `countdown(1)` etc.
- Define the **generic formula** for repeating the method/function i.e. `countDown (n-1)`,
`System.out.print (11-n)`
- Define the **base case / termination condition** to stop the recursive method. i.e.
`n == 0`, `n == 11`

Recursion – Code Structure

```
if (!(terminating condition) ) {  
    Take a step closer to terminating condition  
    Call method RECURSIVELY on smaller subproblem  
}  
  
if (terminating condition) {  
    DO FINAL ACTION  
}  
else {  
    Take one step closer to terminating condition  
    Call method RECURSIVELY on smaller subproblem  
}
```

Factorial

Example 02

C

P

C

S

2

0

4

Factorial – Generic Formula

- $5! = 5 * 4 * 3 * 2 * 1$
 $= 5 * (4 * 3 * 2 * 1)$
 $= 5 * 4!$

- $4! = 4 * 3 * 2 * 1$
 $= 4 * (3 * 2 * 1)$
 $= 4 * 3!$

- $3! = 3 * 2 * 1$
 $= 3 * (2 * 1)$
 $= 3 * 2!$

- Thus we observed

- $5! = 5 * 4!$

- $4! = 4 * 3!$

- $3! = 3 * 2!$

- So, for any value “n”

- $n! = n * (n - 1) !$

C

P

C

S

2

0

4

Factorial – **Base Case**

- The factorial of 0 and 1 is always 1 (one)
- $1! = 1$

OR

- $0! = 1$

C

P

C

S

2

0

4

Factorial – Code

- Generic Formula

$$n! = n * (n - 1) !$$

- Base Case

$$0! = 1$$

- Initial Method Call

factorial(n);

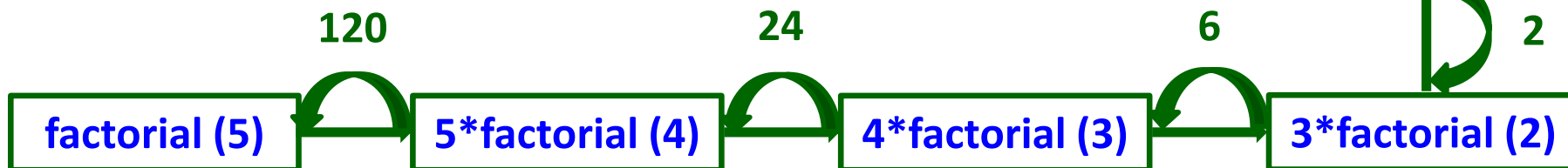
```
public static int factorial(int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * factorial(n - 1);  
}
```

Factorial – Code Tracing

factorial(5)

Result = 120

```
public static int factorial(int n) {
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
```



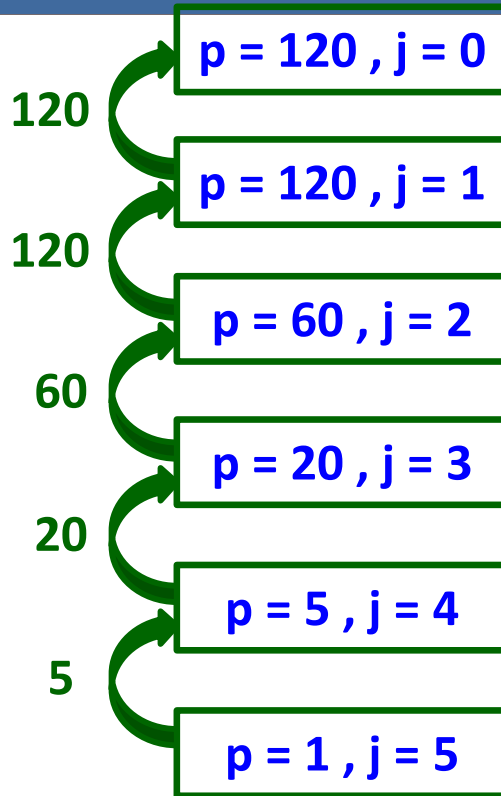
Iterative vs Recursive

Introduction

Return 120

“No Bonus” marks for this course anymore

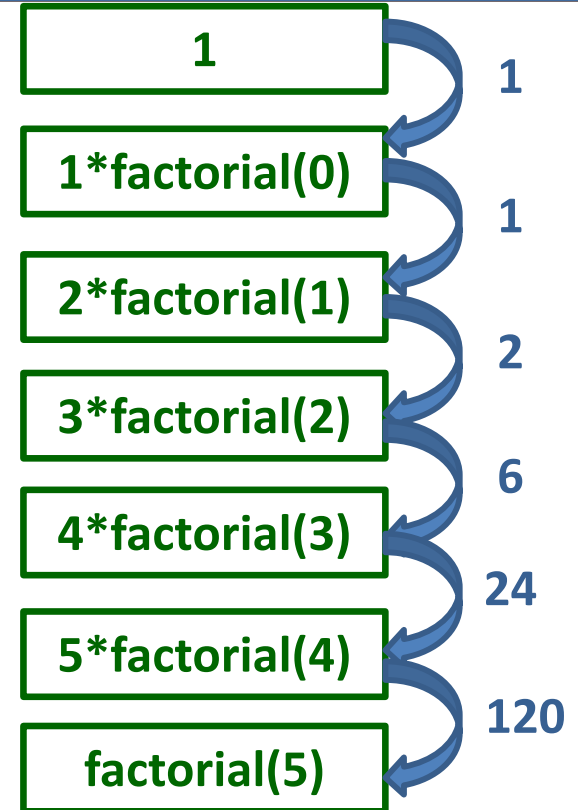
C
P
C
S
2
0
4



B
o
t
t
o
m
-
U
p

VS

T
o
p
-
D
o
w
n



Return 120

```

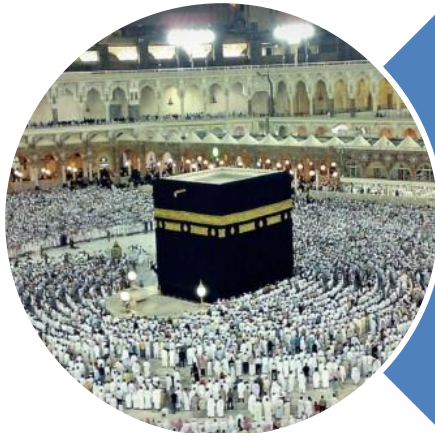
public static int fact(int n){
    int p, j;
    p = 1;
    for ( j=n; j>=1; j--)
        p = p* j;
    return ( p );
}
    
```

ITERATIVE

```

public static int factorial(int n) {
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
    
```

RECURSIVE



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**